

Arithmetic Expression Evaluator

Test Case

Version 1.0

Arithmetic Expression evaluator	Version: 1.0
Test Case	Date: 05/05/2026
EECS348-G10-TC	

Revision History

Date	Version	Description	Author
05/05/2026	1.0	Section 1,2,3,4,5,6,7 complete	Ayushma Koirala

Arithmetic Expression evaluator	Version: 1.0
Test Case	Date: 05/05/2026
EECS348-G10-TC	

Table of Contents

1. Purpose	4
2. Test case identifier	4
3. Test item	4
4. Input specifications	4
5. Output specifications	4
6. Environmental needs	4
6.1.1 Hardware	4
6.1.2 Software	4
6.1.3 Other	4
7. Special procedural requirements	5
8. Intercase dependencies	5

Arithmetic Expression evaluator	Version: 1.0
Test Case	Date: 05/05/2026
EECS348-G10-TC	

Test Case

1. Purpose

This Test Case Specification document defines a test case for the Arithmetic Expression Evaluator that should be tested. The test cases confirm that the system correctly accepts arithmetic expressions, tokenizes and parses them, evaluates them according to PEMDAS and associativity rules, and returns either the correct numeric result or a clear error message for invalid input.

The tests cover valid integer and floating-point expressions, supported operators, unary operators, parentheses, exponentiation, modulo, and expected error handling behavior such as invalid characters, missing operands, mismatched parentheses, and division by zero.

2-5. Test case identifier, Test item, Input specification, Output specifications

The following table lists the main test cases for the Arithmetic Expression Evaluator. Each case includes a unique identifier, the feature being tested, the exact input expression, and the expected output or behavior.

2. Test case ID	3. Test Item	4. Input specification	5. Output Specification	Pass/ Fail
TC01	Basic addition and token recognition	5+2	7	
TC02	Basic subtraction	6-3	3	
TC03	Multiplication precedence over addition	1+2*3	7	
TC04	Parentheses overriding precedence	(4+5) *6	54	
TC05	Division operation	14/7	7	
TC06	Modulo operation	15 % 8	7	
TC07	Exponentiation operation	3**3	9	
TC08	Right-to-left exponent associativity	2**3**2	512	
TC09	Left-to-right associativity for same-precedence operators	10-1-4	5	
TC10	Unary negative number	-5+2	-3	

Arithmetic Expression evaluator	Version: 1.0
Test Case	Date: 05/05/2026
EECS348-G10-TC	

TC11	Unary positive number	+7*2	14	
TC12	Floating-point input	1.5+1.5	3	
TC13	Scientific notation floating-point input	1e2+5	105	
TC14	Whitespace handling	8+2	10	
TC15	Nested parentheses	((2+3)*(4-1))	15	
TC16	Invalid character detection	10+a	Error message indicating invalid character/token at or near a	
TC17	Division by zero	1/0	Error message indicating division by zero	
TC18	Mismatched parentheses	(2+3	Error message indicating mismatched parenthesis	
TC19	Missing operand	4+	Error message indicating missing operand or incomplete expression	
TC20	Incorrect operator sequence	5** *2	Error message indicating invalid operator sequence or missing operand	

6. Environmental needs

6.1.1 Hardware

No special hardware is required. The test cases can be executed on any computer capable of compiling and running a C++ program.

6.1.2 Software

The program should be compiled using a C++17 compliant compiler such as g++. The test environment should include a command-line terminal for entering expressions and viewing output. No third-party libraries, APIs, or external software frameworks are required beyond the C++ standard library.

Arithmetic Expression evaluator	Version: 1.0
Test Case	Date: 05/05/2026
EECS348-G10-TC	

6.1.3 Other

The latest version of the project source code should be pulled from the shared GitHub repository before testing. Testers should record actual output and mark each test as pass or fail based on whether the actual output matches the expected output.

7. Special procedural requirements

Before executing the test cases, the tester should compile the program successfully and run it from the command line. Each test input should be entered exactly as shown in the table. For error cases, the exact wording of the error message may vary, but the message must clearly identify the correct error type and should not cause the program to crash. After each failed test, the defect should be documented in GitHub or the team's defect log, corrected by the development team, and retested.

8. Intercase dependencies

There are no strict intercase dependencies. However, it is recommended that the basic valid-expression tests, such as TC01 through TC07, be executed before the more complex precedence, associativity, nested-parentheses, and error-handling cases. This order helps confirm that the Tokenizer, Parser, Evaluator, and Error Handler are working correctly at a basic level before testing more complex behavior.